LNS: Approximation Methods and Extended Precision Formats

Henrique Teixeira

2026-03-26

Abstract

This document extends previous work on the Logarithmic Number System (LNS) arithmetic unit for RISC-V, which demonstrated a 16-bit LNS format achieving IEEE-754 equivalent precision for neural network weights using linear spline approximation with a greedy algorithm, requiring only 324 bytes of memory and synthesizing at 125 MHz on FPGA.

The present work broadens this foundation in two directions. First, it revisits lns8 Q4.3 and lns16 Q8.7 with a systematic comparison of approximation methods beyond linear splines, including Newton's divided differences, minimax polynomial approximation via the Remez algorithm, and piecewise Chebyshev approximation. Second, it extends the LNS framework to lns32 Q16.15 and lns64 Q32.31, where the domain of the correction functions $f^+(x) = \log_2(1 + 2^x)$ and $f^-(x) = \log_2(1 - 2^x)$ grows significantly and spline table sizes become impractical, motivating the transition to compact polynomial approximations.

Acknowledgements

We would like to thank our supervisor Nuno Paulino and co-supervisors Luís Sousa and Guilherme Oliveira for the help and guidance during this work.

Table of Contents

List of Figures

List of Tables

List of Acronyms

NN	Neural Network
FP	Floating Point
LNS	Logarithmic Number System
LNSU	Logarithmic Number System Unit
LUT	Look Up Table
DSP	Digital Signal Processing block/unit
XF	Spline format storing (x, f) point pairs
XMB	Spline format storing (x, m, b) segment coefficients
NDD	Newton's Divided Differences
MAE	Maximum Absolute Error
MRE	Maximum Relative Error
RMSE	Root Mean Square Error

1 Introduction

The Logarithmic Number System (LNS) offers a compelling alternative to IEEE-754 floating point. By encoding values as fixed-point binary logarithms, multiplication and division reduce to integer addition and subtraction, and square root reduces to a single arithmetic right shift. These properties eliminate the need for mantissa multipliers in hardware, significantly reducing area and power consumption.

Previous work demonstrated an lns16 Q8.7 arithmetic unit embedded in a RISC-V core, achieving a maximum one-bit approximation error for addition and subtraction using linear splines generated by a greedy algorithm, with a total memory footprint of 324 bytes. The design synthesized at 125 MHz on a Pynq Z2 FPGA with single-cycle latency.

This document builds on that foundation. The first goal is to evaluate alternative approximation methods for the correction functions required by LNS addition and subtraction on lns8 and lns16 formats, characterising the trade-off between approximation accuracy, evaluation cost, and memory footprint. The second goal is to extend the LNS framework to 32-bit and 64-bit formats, where the growing domain of the correction functions makes the linear spline approach impractical and more compact polynomial representations become necessary.

1.1 Scope

This document covers:

- A review of the LNS format and the derivation of the correction functions for add and sub operations.
- A summary of the previous linear spline (XF and XMB) approach and its results for lns8 and lns16.
- An analysis of the correction functions' behaviour at increased precision and wider domains (lns32, lns64).
- A survey and analysis of candidate approximation methods: Newton's divided differences, minimax polynomial approximation and piecewise Chebyshev approximation.
- A comparison framework for evaluating approximation methods across all four formats.
- Open questions and planned implementation work.

2 Background

2.1 The Logarithmic Number System

In LNS, a non-zero real number x is represented by a sign bit S_x and a fixed-point binary logarithm L_x:

$$\begin{aligned} S_x &= \begin{cases} 1, \text{if } x < 0 \\ 0, \text{if } x \geq 0 \end{cases} \\ L_x &= \log_2(|x|) \\ x &= (-1)^{S_x} \cdot 2^{L_x} \end{aligned} \tag{1}$$

L_x is a two's complement fixed-point number. Negative values of L_x represent magnitudes in $(0, 1)$ and positive values represent magnitudes in $(1, +\infty)$.

2.2 Supported Formats

The previous supported LNS formats use one sign bit and a fixed-point exponent split into integer and fractional parts:

Format	S_x	Integer bits	Fractional bits	Value range
lns8 Q4.3	1	4	3	2^{-8} to $2^{7.875}$
lns16 Q8.7	1	8	7	2^{-128} to $2^{127.992}$

Table 1: LNS format specifications

The fractional resolution per step is 2^{-F} where F is the number of fractional bits, giving a relative quatization error of $2^{2^{-F}} - 1$. For lns8 this is $2^{-3} = 0.125$, with a relative quantization error per step of approximately $2^{\frac{1}{8}} - 1 \approx 9\%$. For lns16 it is $2^{-7} = 0.0078$, approximately $2^{\frac{1}{128}} - 1 \approx 0.54\%$.

2.3 Arithmetic Operations

The operations in LNS follow directly from the logarithmic representation:

$$\begin{aligned} z = x \cdot y &\Leftrightarrow L_z = L_x + L_y \\ z = \frac{x}{y} &\Leftrightarrow L_z = L_x - L_y \\ z = \sqrt{x} &\Leftrightarrow L_z = L_x \gg 1 \\ z = x + y &\Leftrightarrow L_z = L_x + \log_2\left(1 + 2^{L_y - L_x}\right), \quad |x| \geq |y| \\ z = x - y &\Leftrightarrow L_z = L_x + \log_2\left(1 - 2^{L_y - L_x}\right), \quad |x| \geq |y| \end{aligned} \tag{2}$$

Multiplication, division, and square root are exact up to quantization. Addition and subtraction require evaluating the correction functions:

$$\begin{aligned} f^+(x) &= \log_2(1 + 2^x), \quad x \in (-\infty, 0] \\ f^-(x) &= \log_2(1 - 2^x), \quad x \in (-\infty, 0) \end{aligned} \tag{3}$$

where $x = L_y - L_x \leq 0$ is enforced by swapping operands if necessary so that the larger magnitude is always taken as the base.

This comes from the following derivation:

- Given $|x| > |y|$:

$$\begin{aligned} z &= x \pm y \\ L_z &= \log_2\left(2^{L_x} \pm 2^{L_y}\right) \\ L_z &= \log_2\left(2^{L_x}\left(1 \pm 2^{L_y - L_x}\right)\right) \\ L_z &= L_x + \log_2\left(1 \pm 2^{L_y - L_x}\right) \end{aligned} \tag{4}$$

2.4 Properties of the Correction Functions

2.4.1 Addition: $f^+(x) = \log_2(1 + 2^x)$

f^+ is smooth and bounded on $(-\infty, 0]$. It increases monotonically from 0 at $-\infty$ to 1 at $x = 0$. Its first and second derivative are:

$$\begin{aligned} (f^+)'(x) &= \frac{2^x}{1 + 2^x} \\ (f^+)''(x) &= \ln(2)\frac{2^x}{(1 + 2^x)^2} \end{aligned} \tag{5}$$

Both are always positive, monotonically increasing, and approach 0 at $-\infty$ and $\frac{1}{2}$ and $\frac{\ln(2)}{4}$ at $x = 0$, respectively. These derivatives will be important to derive upper bound expressions for the error functions.

2.4.2 Subtraction: $f^-(x) = \log_2(1 - 2^x)$

f^- is defined on $(-\infty, 0)$ only. As $x \to 0^-$, $f^-(x) \to -\infty$ due to the singularity of $\log_2$ at zero. Its first and second derivative are:

$$\begin{aligned} (f^-)'(x) &= -\frac{2^x}{1 - 2^x} \\ (f^-)''(x) &= -\ln(2)\frac{2^x}{(1 - 2^x)^2} \end{aligned} \tag{6}$$

These are always negative and diverge to $-\infty$ as $x \to 0^-$. The unbounded second derivative near $x = 0$ is the fundamental reason that the subtraction function requires significantly more intervals than the addition function to achieve the same approximation error. Any approximation method must handle this singularity explicitly, either by excluding a neighbourhood of 0 and clamping, or by using a change of variable that regularises the behaviour.

3 Previous Work: Linear Splines

3.1 XF and XMB Representations

The previous work approximated f^+ and f^- with piecewise linear splines generated by a greedy algorithm. Given a budget of $n+1$ points $\{(x_i, f_i)\}_{\{i=0\}}^{\{n\}}$, the standard linear spline interpolation formula for an interval $[x_{i-1}, x_i]$ is:

$$\mathrm{LS}_i(x) = \frac{(x_i - x)f_{i-1} + (x - x_{i-1})f_i}{h_i}, \quad h_i = x_i - x_{i-1} \tag{7}$$

This is the XF representation. It stores (x_i, f_i) pairs and evaluates the weighted average at query time. An equivalent form is:

$$\mathrm{LS}_i(x) = m_i \cdot x + b_i, \quad m_i = \frac{f_i - f_{i-1}}{h_i}, \quad b_i = \frac{f_{i-1}x_i - f_i x_{i-1}}{h_i} \tag{8}$$

The error upper bound formula for a linear spline is:

$$\varepsilon \leq \frac{1}{8} \max_{[a,b]} |f''(x)| \; h_i^2 \tag{9}$$

This is the XMB representation. It precomputes the slope m_i and intercept b_i per segment, reducing runtime evaluation to a single multiply-add. The trade-off is increased memory: XF requires $4(n+1)$ bytes for lns16 (two 2-byte fields per point), while XMB requires $6(n+1)$ bytes (three 2-byte fields).

3.2 Greedy Algorithm

The greedy algorithm starts with the two boundary points and iteratively adds the midpoint of the interval with the largest error estimate:

$$\varepsilon_i \leq \frac{\ln(2) \cdot 2^{x_i} \cdot h_i^2}{8(1 \pm 2^{x_i})^2} \tag{10}$$

Since the error bound is monotonically increasing for both functions (the maximum of $|f''|$ on each interval is at the right endpoint), the interval to split is always the one with the largest h_i weighted by the local curvature. The minimum allowed interval length is 2 (in fixed-point units), enforcing that consecutive spline points are distinct in the integer representation.

3.3 Results for lns8 and lns16

Format	Method	Operation	Error (bits)	Table size (bytes)
lns8 Q4.3	XF	Add	1	16
		Sub	1	48
		Add+Sub	1	64
	XMB	Add	1	24
		Sub	1	42
		Add+Sub	1	66
lns16 Q8.7	XF	Add	1	40
		Sub	1	200
		Add+Sub	1	240
	XMB	Add	1	72
		Sub	1	252
		Add+Sub	1	324

Table 2: Linear spline results for lns8 Q4.3 and lns16 Q8.7

The XF tables achieve lower memory for the same error, while XMB reduces runtime to a single multiply-add plus a table lookup, enabling single-cycle latency.

The subtraction table requires substantially more rows than the addition table, because the second derivative of f^- diverges near $x = 0$, requiring dense sampling near the singularity.

3.4 Limitations at Higher Precision

The linear spline approach works well for lns8 and lns16, because the domain of the correction functions is small − the argument $x = L_y - L_x$ ranges over at most 8 or 16 bits, respectively. For lns32 and lns64, the domain grows proportionally with the exponent width, and the number of intervals required to maintain a fixed error bound scales roughly as $O\left(\sqrt{\frac{1}{\varepsilon}}\right) = O\left(\sqrt{\frac{1}{2^{-n}}}\right) = O\left(2^{\frac{n}{2}}\right)$ per unit of domain, making table-based approaches impractical.

Additionally, even for lns8 and lns16 there is room to improve approximation quality beyond piecewise linear. Higher-degree polynomial approximations can achieve the same error with fewer segments or lower error with the same number of segments.

4 Candidate Approximation Methods

This chapter surveys the methods under consideration for replacing or supplementing linear splines for lns8, lns16, and for use in lns32 and lns64.

4.1 Error Formula

Theorem 1 Lagrange Interpolation Error Formula

Given a function $f(x) \in C^{n+1}[a,b]$, let $p_n(x) \in \mathbb{P}_n$ be the interpolative polynomial of $f(x)$ in the unique x's $\{x_i \mid 0 \le i \le n\}$, then $\forall x \in [a,b], \exists c_x \in [a,b]$:

$$\varepsilon = f(x) - p_n(x) = \frac{1}{(n+1)!} f^{(n+1)}(c_x)\pi_{n+1}(x) \tag{11}$$

where $\pi_{n+1}(x) = \prod_{k=0}^{n}(x - x_k)$

4.1.1 f^+ and f^- Error Upper Bound

Given $x < 0$:

$$\begin{aligned}
\log_2(1 \pm 2^x) &= \frac{1}{\ln(2)} \ln(1 \pm 2^x) \\
&= \frac{1}{\ln(2)} \int_{-\infty}^{x} \frac{\mathrm{d}}{\mathrm{d}t}(\ln(1 \pm 2^t))\,\mathrm{d}t \\
&= \frac{1}{\ln(2)} \int_{-\infty}^{x} \frac{1}{1 \pm 2^t} \frac{\mathrm{d}}{\mathrm{d}t}(1 \pm 2^t)\,\mathrm{d}t \\
&= \frac{\pm}{\ln(2)} \int_{-\infty}^{x} \frac{2^t}{1 \pm 2^t}\,\mathrm{d}t
\end{aligned}$$

As $\pm 2^t < 0, \forall t \in (-\infty, x], x < 0$, then $\frac{1}{1 \pm 2^t}$ can be expanded using the geometric series infinite series representation $\frac{1}{1-r} = \sum_{i \ge 0} r^n$.

$$= \frac{\pm}{\ln(2)} \int_{-\infty}^{x} 2^t \sum_{n \ge 0} (\mp 2^t)^n\,\mathrm{d}t$$

Since the series converges, the swap of the integration and sumation operations are justified.

$$\begin{aligned}
&= \frac{\pm}{\ln(2)} \sum_{n \ge 0} (\mp 1)^n \int_{-\infty}^{x} 2^{(n+1)t}\,\mathrm{d}t \\
&= \frac{1}{\ln(2)} \sum_{n \ge 0} (\mp 1)^{n-1} \left(\frac{1}{\ln(2)(n+1)} 2^{(n+1)t} \right) \Bigg|_{t \to -\infty}^{t=x} \\
&= \frac{1}{\ln(2)^2} \sum_{n \ge 0} \frac{(\mp 1)^{n-1}}{n+1} 2^{(n+1)x} \\
&= \frac{1}{\ln(2)^2} \sum_{n \ge 1} \frac{(\mp 1)^{n}}{n} 2^{nx}
\end{aligned}$$

And so, we finnaly get an infinite series representation of f^+ and f^-.

$$\log_2(1 \pm 2^x) = \frac{1}{\ln(2)^2} \sum_{n \geq 1} \frac{(\mp 1)^n}{n} 2^{nx}, x < 0 \tag{12}$$

Now, we can use this infinite series representation to calculate an upper bound for the k-th derivative of both functions:

$$\frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1 \pm 2^x) = \frac{1}{\ln(2)^2} \frac{\mathrm{d}^k}{\mathrm{d}x^k} \sum_{n \geq 1} \frac{(\mp 1)^n}{n} 2^{nx}$$

The switch of the summation and derivative operators is also justified as the series does converge.

$$\frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1 \pm 2^x) = \frac{1}{\ln(2)^2} \sum_{n \geq 1} \frac{(\mp 1)^n}{n} \frac{\mathrm{d}^k}{\mathrm{d}x^k} 2^{nx}$$

$$\frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1 \pm 2^x) = \ln(2)^{k-2} \sum_{n \geq 1} (\mp 1)^n n^{k-1} 2^{nx}$$

If we can find a term a, such that all terms $n^{k-1}2^{nx} \leq a^n$, then the sum of the terms will be larger than our current series and we have a simple upper bound we can compute.

Following the Alternative Series Estimation Error for f^+,
as $|a_{n+1}| < |a_n| \wedge \lim_{n \to \infty} a_n = 0, a_n = (-1)^n n^{k-1} 2^{nx}$:

$$
\begin{aligned}
|a_{n+1}| &< |a_n| \\
|(n+1)^{k-1}2^{(n+1)x}| &< |n^{k-1}2^{nx}| \\
\left(1+\frac{1}{n}\right)^{k-1} &< 2^{-x} \\
1+\frac{1}{n} &< 2^{-\frac{x}{k-1}} \\
\frac{1}{n} &< 2^{-\frac{x}{k-1}}-1 \\
n &> \frac{1}{2^{-\frac{x}{k-1}}-1}
\end{aligned}
$$

$$
|R_n| < |a_{n+1}|
$$

$$
\left|\sum_{n\geq\left\lceil\frac{1}{2^{-\frac{x}{k-1}}-1}\right\rceil}(-1)^n n^{k-1}2^{nx}\right| < \left|\left(\left\lceil\frac{1}{2^{-\frac{x}{k-1}}-1}\right\rceil+1\right)^{k-1}2^{(1+1)x}\right|
$$

$$
\begin{aligned}
&= \left(\left\lceil\frac{1}{2^{-\frac{x}{k-1}}-1}\right\rceil+1\right)^{k-1}2^{2x} \\
&= \left(\left\lceil\frac{1}{1-2^{\frac{x}{k-1}}}-1\right\rceil+1\right)^{k-1}2^{2x}
\end{aligned}
$$

$$
\begin{aligned}
\left\lceil\frac{1}{1-2^t}-1\right\rceil+1 &= \left\lceil\frac{2^t}{1-2^t}\right\rceil+1 \\
&\leq \frac{2^t}{1-2^t}+2 \\
&= \frac{2^t+2-2\cdot 2^t}{1-2^t} \\
&= \frac{2-2^t}{1-2^t} \\
&= 2\frac{1-2^{t-1}}{1-2^t}
\end{aligned}
$$

$$
\begin{aligned}
x<0 &\rightarrow \frac{x}{k-1}<0 \\
&\rightarrow t<0
\end{aligned}
$$

$$
2\frac{1-2^{t-1}}{1-2^t} \leq \frac{1}{1-2^t}
$$

$$
K = \left|\sum_{n\geq 1}^{n=\left\lceil\frac{1}{1-2^{\frac{x}{k-1}}}-1\right\rceil-1}(-1)^n n^{k-1}2^{nx}\right|
$$

By the Abel-Plana formula:

$$\sum_{n\geq 0}(-1)^n f(n) = \frac{1}{2}f(0) + \int_0^\infty \frac{f(it)-f(-it)}{2i\sinh(\pi t)}\,\mathrm{d}t$$

$$\begin{aligned}
0\cdot 2^0 + \sum_{n\geq 1}(-1)^n n^{k-1}2^{nx} &= 0 + \int_0^\infty \frac{(it)^{k-1}2^{itx} - (-it)^{k-1}2^{-itx}}{2i\sinh(\pi t)}\,\mathrm{d}t, k>0\\
&= -i^k\int_0^\infty t^{k-1}\frac{e^{\ln(2)itx} - (-1)^{k-1}e^{-\ln(2)itx}}{2\sinh(\pi t)}\,\mathrm{d}t\\
&= -i^k\left(\int_0^\infty t^{k-1}\frac{\cos(\ln(2)xt)}{\sinh(\pi t)}\,\mathrm{d}t \vee \int_0^\infty t^{k-1}\frac{\sin(\ln(2)xt)}{\sinh(\pi t)}\,\mathrm{d}t\right)
\end{aligned}$$

$$\begin{aligned}
\int_0^\infty t^{k-1}\frac{\cos(at)}{\sinh(\pi t)}\,\mathrm{d}t &= \int_0^\infty t^{k-1}\frac{\cos(at)}{e^{\pi t}-e^{-\pi t}}\,\mathrm{d}t\\
&= \int_0^\infty t^{k-1}e^{-\pi t}\frac{\cos(at)}{1-e^{-2\pi t}}\,\mathrm{d}t\\
&= \int_0^\infty t^{k-1}e^{-\pi t}\cos(at)\sum_{n\geq 0}e^{-2\pi nt}\,\mathrm{d}t\\
&= \sum_{n\geq 0}\int_0^\infty t^{k-1}\Re(e^{iat})e^{-(2n+1)\pi t}\,\mathrm{d}t\\
&= \frac{1}{2}\Re\left(\sum_{n\geq 0}\int_0^\infty t^{k-1}e^{(-\pi(2n+1)+ia)t}\,\mathrm{d}t\right)\\
&= \frac{1}{2}\Re\left(\sum_{n\geq 0}\frac{1}{(\pi(2n+1)-ia)^k}\int_0^\infty z^{k-1}e^{-z}\,\mathrm{d}z\right)\\
&= \frac{\Gamma(k)}{2}\Re\left(\sum_{n\geq 0}\frac{1}{\left(\left(n+\frac{1-ia}{2\pi}\right)2\pi\right)^k}\right)\\
&= \frac{\Gamma(k)}{2(2\pi)^k}\Re\left(\sum_{n\geq 0}\frac{1}{\left(n+\frac{1-ia}{2\pi}\right)^k}\right)\\
&= \frac{\Gamma(k)}{2(2\pi)^k}\Re\left(\sum_{n\geq 0}\frac{1}{\left(n+\frac{1}{2\pi}-i\frac{a}{2\pi}\right)^k}\right)\\
&= \frac{\Gamma(k)}{2(2\pi)^k}\Re\left(\zeta\left(k,\frac{1}{2\pi}-i\frac{a}{2\pi}\right)\right)
\end{aligned}$$

$$\int_0^\infty t^{k-1}\frac{\sin(at)}{\sinh(\pi t)}\,\mathrm{d}t = \frac{\Gamma(k)}{2(2\pi)^k}\Im\left(\zeta\left(k,\frac{1}{2\pi}-i\frac{a}{2\pi}\right)\right)$$

$$\sum_{n\geq 1}(-1)^n n^{k-1}2^{nx} = \begin{cases} -i^k\dfrac{\Gamma(k)}{(2\pi)^k}\Im\left(\zeta\left(k,\dfrac{1}{2}-i\dfrac{\ln(2)}{2}x\right)\right) & \text{if } k \text{ odd}\\ -i^k\dfrac{\Gamma(k)}{2(2\pi)^k}\Re\left(\zeta\left(k,\dfrac{1}{2\pi}-i\dfrac{\ln(2)x}{2\pi}\right)\right) & \text{otherwise}\end{cases}$$

As $\zeta(s,z)$ satisfies the following property $\zeta(s,\overline{z}) = \overline{\zeta(s,z)}$:

$$\sum_{n\geq 1}(-1)^n n^{k-1}2^{nx} = \begin{cases} -(-1)^{\frac{k-1}{2}}\dfrac{\Gamma(k)}{(2\pi)^k}\cdot i\cdot\Im\left(\zeta\left(k,\dfrac{1}{2\pi}-i\dfrac{\ln(2)}{2\pi}x\right)\right) & \text{if } k \text{ odd} \\ -(-1)^{\frac{k}{2}}\dfrac{\Gamma(k)}{2(2\pi)^k}\Re\left(\zeta\left(k,\dfrac{1}{2\pi}-i\dfrac{\ln(2)}{2\pi}x\right)\right) & \text{otherwise} \end{cases}$$

$$\begin{aligned}
\left|\sum_{n=0}(-1)^n n^{k-1}2^{nx}\right| &\leq \left|\int_0^\infty t^{k-1}\frac{1}{\sinh(\pi t)}\,\mathrm{d}t\right| \\
&\leq \left|\int_0^\infty t^{k-1}\frac{2}{e^{\pi t}-e^{-\pi t}}\,\mathrm{d}t\right| \\
&\leq 2\left|\int_0^\infty t^{k-1}\frac{e^{-\pi t}}{1-e^{-2\pi t}}\,\mathrm{d}t\right| \\
&\leq 2\left|\int_0^\infty t^{k-1}e^{-\pi t}\sum_{m\geq 0}e^{-2\pi m t}\,\mathrm{d}t\right| \\
&\leq 2\left|\int_0^\infty \sum_{m\geq 0}t^{k-1}e^{-(2m+1)\pi t}\,\mathrm{d}t\right| \\
&\leq 2\left|\sum_{m\geq 0}\int_0^\infty t^{k-1}e^{-(2m+1)\pi t}\,\mathrm{d}t\right| \\
&\leq 2\left|\sum_{m\geq 0}\int_0^\infty \left(\frac{z}{(2m+1)\pi}\right)^{k-1}e^{-z}\frac{\mathrm{d}z}{(2m+1)\pi}\right| \\
&\leq 2\left|\sum_{m\geq 0}\frac{1}{((2m+1)\pi)^k}\int_0^\infty z^{k-1}e^{-z}\,\mathrm{d}z\right| \\
&\leq 2\left|\frac{\Gamma(k)}{\pi^k}\left(\sum_{m\geq 1}\frac{1}{m^k}-\sum_{m\geq 1}\frac{1}{(2m)^k}\right)\right| \\
&\leq 2\left|\frac{\Gamma(k)}{\pi^k}\left(\zeta(k)-2^{-k}\sum_{m\geq 1}\frac{1}{m^k}\right)\right| \\
&\leq 2\Gamma(k)\pi^{-k}\zeta(k)\left(1-2^{-k}\right)
\end{aligned}$$

Therefore, the k-th derivative of the f^+ is bounded by:

$$\frac{\mathrm{d}^k}{\mathrm{d}x^k}\log_2(1-2^x) \leq \ln(2)^{k-2}\left(K+\left(1-2^{\frac{x}{k-1}}\right)^{1-k}2^{2x}\right)$$

Following D'Alambert Criteria for f^-, as $\lim_{n\to\infty}\frac{a_{n+1}}{a_n} = L, a_n = \frac{2^{nx}}{n^{k+1}}$:

$$\begin{aligned}\lim_{n\to\infty}\frac{(n+1)^{k-1}2^{(n+1)x}}{n^{k-1}2^{nx}} &= \lim_{n\to\infty}2^{(n+1)x-nx}\left(\frac{n+1}{n}\right)^{k+1}\\ &= 2^x\lim_{n\to\infty}\left(1+\frac{1}{n}\right)^{k+1}\\ &= 2^x\end{aligned}$$

$$\begin{aligned}L<1 &\Leftrightarrow 2^x<1\\ &\Leftrightarrow x<0\end{aligned}$$

$$R_n \le a_{n+1}\frac{1}{1-L}$$

$$\begin{aligned}\sum_{n\ge1}\frac{2^{nx}}{n^{k+1}} &\le 2^{k-1}2^{2x}\frac{1}{1-2^x}\\ &= \frac{2^{2x+k-1}}{1-2^x}\end{aligned}$$

Therefore, the k-th derivative of the f^- is bounded by:

$$\frac{\mathrm{d}^k}{\mathrm{d}x^k}\log_2(1-2^x) \le \ln(2)^{k-2}\frac{2^{2x+k-1}}{1-2^x}$$

Now we can use the Lagrange Interpolation Error Formula with $n+1$ points.

$$\varepsilon^+ \le \max_{x\in[a,b]}\frac{1}{(n+1)!}\ln(2)^{(n+1)-2}\left(K+\left(1-2^{\frac{x}{(n+1)-1}}\right)^{1-(n+1)}2^{2x}\right)\left|\pi_{n+1}(x)\right|$$

$$\varepsilon^- \le \max_{x\in[a,b]}\frac{1}{(n+1)!}\ln(2)^{(n+1)-2}\frac{2^{2x+(n+1)-1}}{1-2^x}\left|\pi_{n+1}(x)\right|$$

For an interval $[a,b]$, $\{x_i \mid 0\le i\le n\}\in[a,b]$, where n is the degree of the polynomial on the interval $[a,b]$ and $x\in[a,b]$ and $x<0$:

$$\varepsilon_n^+ \le \max_{x\in[a,b]}\frac{1}{(n+1)!}\ln(2)^{n-1}\left(K+\left(1-2^{\frac{x}{n}}\right)^{-n}2^{2x}\right)(b-a)^{n+1}$$

$$\varepsilon_n^- \le \max_{x\in[a,b]}\frac{1}{(n+1)!}\ln(2)^{n-1}\frac{2^{2x+n}}{1-2^x}(b-a)^{n+1}$$

$$\varepsilon_n^+ \le \frac{1}{(n+1)!}\ln(2)^{n-1}\left(K+\left(1-2^{\frac{x}{n}}\right)^{-n}2^{2x}\right)(b-a)^{n+1}$$

$$\varepsilon_n^- \le \frac{1}{(n+1)!}\ln(2)^{n-1}\frac{2^{2b+n}}{1-2^b}(b-a)^{n+1}$$

Splitting the functions into multiple branches with intervals $\{[x_{i-1},x_i] \mid 0\le i\le k\}$ with degree n_i:

$$\begin{aligned}\varepsilon_{n_i}^+ &\le \frac{1}{(n_i+1)!}\ln(2)^{n_i-1}\left(K+\left(1-2^{\frac{x}{n_i}}\right)^{-n_i}2^{2x}\right)h_i^{n_i+1}\\ \varepsilon_{n_i}^- &\le \frac{1}{(n_i+1)!}\ln(2)^{n_i-1}\frac{2^{2x_i+n_i}}{1-2^{x_i}}h_i^{n_i+1}\end{aligned} \tag{13}$$

Now, we have an upper bound that we can use to modulate the approximation error of both functions in all k sections.

But we can go further. Let p be the precision of our LNS format, that is, LNSn-Q$i.p$, we want $\varepsilon_{n_i}\le 2^{-p}$:

$$2^{-p^+} \leq \frac{1}{(n_i+1)!}\ln(2)^{n_i-1}h_i^{n_i+1}\left(K+\left(1-2^{\frac{x}{n}}\right)^{-n_i}2^{2x}\right)$$

$$p^+ \geq \log_2\left((n_i+1)!\cdot\ln(2)^{1-n_i}\cdot h_i^{-(n_i+1)}\cdot 2^{-2x_i-n_i}\right)$$

$$p^+ \geq \log_2((n_i+1)!) + (1-n_i)\log_2(\ln(2)) - (n_i+1)\log_2(h_i) + \log_2(2^{-2x_i-n_i})$$

$$p^+ \geq \log_2((n_i+1)!) + (n_i+1)(\log_2(h_i^{-1}) - \log_2(\ln(2)) - 1) - 2x_i + 2\log_2(\ln(2))$$

$$2^{-p^-} \leq \frac{1}{(n_i+1)!}\ln(2)^{n_i-1}h_i^{n_i+1}2^{2x_i+n_i}\frac{1}{1-2^{x_i}}$$

$$p^- \geq \log_2\left((n_i+1)!\cdot\ln(2)^{1-n_i}\cdot h_i^{-(n_i+1)}\cdot 2^{-2x_i-n_i}\cdot(1-2^{x_i})\right)$$

$$p^- \geq \log_2((n_i+1)!) + (1-n_i)\log_2(\ln(2)) - (n_i+1)\log_2(h_i) + \log_2(2^{-2x_i-n_i}) + \log_2(1-2^{x_i})$$

$$p^- \geq \log_2((n_i+1)!) + (n_i+1)(\log_2(h_i^{-1}) - \log_2(\ln(2)) - 1) - 2x_i + \log_2(1-2^{x_i}) + 2\log_2(\ln(2))$$

Finally, we arrive at a relationship between the precision of the format, the degree n_i of the polynomial, the size of the interval, h_i, and, given by the last term, the curvature of the function in that same interval.

$$p^+ \sim \Omega(\log_2((n_i+1)!) + (n_i+1)\log_2(h_i^{-1}) - 2x_i) \tag{14}$$

$$p^- \sim \Omega(\log_2((n_i+1)!) + (n_i+1)\log_2(h_i^{-1}) - 2x_i + \log_2(1-2^{x_i})) \tag{15}$$

4.1.2 f^+ and f^- domain ranges

For an LNS type with precision p, the minimum value in absolute terms is 2^{-p}. So, we want to find $x < 0$, such that $\pm 2^{-p} = \log_2(1 \pm 2^x)$:

$$\begin{aligned} \pm 2^{-p} &= \log_2(1 \pm 2^x) \\ 2^{\pm 2^{-p}} &= 1 \pm 2^x \\ 2^{\pm 2^{-p}} - 1 &= \pm 2^x \\ \pm(2^{\pm 2^{-p}} - 1) &= 2^x \\ x &= \log_2(\pm(2^{\pm 2^{-p}} - 1)) \end{aligned}$$

Type	Precision p (bits)	$x^+_{\min}$	$x^-_{\min}$
LNS8 Q4.3	3	-3.466	-3.591
LNS16 Q8.7	7	-7.525	-7.533
LNS32 Q16.15	15	-15.529	-15.529
LNS32 Q8.23	23	-23.529	-23.529
LNS64 Q33.30	30	-30.529	-30.529
LNS64 Q23.40	40	-40.529	-40.529
LNS64 Q11.52	52	-52.000	-53.000

Table 3: Effective domain lower bound $x^{\pm}_{\min} = \log_2(2^{\pm 2^{-p}} - 1)$ by format and precision

4.1.3 FP to LNS and LNS to FP convertion functions

$$x_{fp} = (-1)^{S_x} \cdot (1 + m_x) \cdot 2^{e_x}$$
$$x_{lns} = (-1)^{S_x} \cdot 2^{L_x}$$

$$x_{fp} = x_{lns}$$
$$(-1)^{S_x} \cdot (1 + m_x) \cdot 2^{e_x} = (-1)^{S_x} \cdot 2^{L_x}$$
$$(1 + m_x) \cdot 2^{e_x} = 2^{L_x}$$
$$\log_2(1 + m_x) + e_x = L_x$$
$$L_x = e_x + \log_2(1 + m_x) \tag{16}$$

$$\log_2(1 + x) = -\frac{1}{\ln(2)} \sum_{n \geq 1} \frac{(-1)^n}{n} x^n, |x| < 1$$
$$= \frac{1}{\ln(2)} \sum_{n=1}^{k} (-1)^n x^n - \frac{1}{\ln(2)} \sum_{n \geq k+1} \frac{(-1)^n}{n} x^n$$

By the Alternaning Series Estimation Theorem:

$$\left| \sum_{n \geq k+1} \frac{(-1)^n}{n} x^n \right| < (-1)^k \frac{x^{k+2}}{k+2} \tag{17}$$

$$\left| \log_2(1 + x) - \frac{1}{\ln(2)} \sum_{n=1}^{k} \frac{(-1)^n}{n} x^n \right| < \frac{|x^{k+2}|}{\ln(2)(k+2)}$$

For the largest x value, $x = 1$, the error of the approximation is bounded by $\frac{1}{\ln(2)(k+2)}$. Therefore, to achieve a precision p:

$$2^{-p} \leq \frac{1}{\ln(2)(k+2)}$$
$$-p \leq -\log_2(\ln(2)(k+2))$$
$$p \geq \log_2(\ln(2)) + \log_2(k+2)$$

The growth of the precision is really slow.

$$
\begin{aligned}
\log_2(1+x) &= \frac{1}{\ln(2)} \sum_{n\geq 1} \frac{(-1)^n}{n} x^n, |x| < 1 \\
\frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1+x) &= \frac{1}{\ln(2)} \frac{\mathrm{d}^k}{\mathrm{d}x^k} \sum_{n\geq 1} \frac{(-1)^n}{n} x^n \\
&= \frac{1}{\ln(2)} \sum_{n\geq k} \frac{(-1)^n}{n} x^{n-k} \\
&= \frac{1}{\ln(2) x^k} \left(\ln(1+x) - \sum_{n\geq 1}^{k-1} \frac{(-1)^n}{n} x^n \right) \\
&= \frac{1}{\ln(2) x^k} \left(\ln(1+x) - \sum_{n\geq 1}^{k-1} \frac{(-1)^n}{n} \int_0^x n t^{n-1} \,\mathrm{d}t \right) \\
&= \frac{1}{\ln(2) x^k} \left(\ln(1+x) - \int_0^x \sum_{n\geq 1}^{k-1} \frac{(-1)^n}{n} n t^{n-1} \,\mathrm{d}t \right) \\
&= \frac{1}{\ln(2) x^k} \left(\ln(1+x) + \int_0^x \sum_{n\geq 1}^{k-1} (-t)^{n-1} \,\mathrm{d}t \right) \\
&= \frac{1}{\ln(2) x^k} \left(\ln(1+x) + \int_0^x \sum_{n\geq 0}^{k-1} (-t)^{n} \,\mathrm{d}t \right) \\
&= \frac{1}{\ln(2) x^k} \left(\ln(1+x) + \int_0^x \frac{1-(-t)^k}{1+t} \,\mathrm{d}t \right) \\
&= \frac{1}{\ln(2) x^k} \left(2\ln(1+x) - (-1)^k \int_0^x \frac{t^k}{1+t} \,\mathrm{d}t \right) \\
&\leq \frac{1}{\ln(2) x^k} \left(2\ln(1+x) - \frac{(-x)^k}{k} \right) \\
&= 2\log_2(1+x) - \frac{(-1)^k}{k} \\
\frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1+x) &\leq 2\log_2(1+x) + \frac{1}{k} \text{ if } k = 0 \bmod 2 \text{ else } 0 \\
\frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1+x) &= \frac{1}{\ln(2)} \sum_{n\geq 0} \frac{(-1)^{n+k}}{n+k} x^k \\
\left| \frac{\mathrm{d}^k}{\mathrm{d}x^k} \log_2(1+x) \right| &\leq \left| \frac{1}{\ln(2)} \frac{(-1)^{k+1}}{k+1} x^k \right|
\end{aligned}
$$

4.2 Newton's Divided Differences

4.2.1 Method

Newton's divided differences construct an interpolating polynomial of degree n through $n+1$ sample points $\{(x_i, f_i) \mid 0 \leq i \leq n\}$. The polynomial is expressed in Newton form:

$$P(x) = \sum_{i=0}^{n} [y_0, ..., y_i] \prod_{j=0}^{i-1} (x - x_j) \tag{18}$$

where $[y_0, ..., y_k]$ denotes the k-th order divided difference, computed recursively as:

$$\begin{aligned} [y_i] &= f(x_i) \\ [y_i, ..., y_{i+k}] &= \frac{[y_{i+1}, ..., y_{i+k}] - [y_i, ..., y_{i+k-1}]}{x_{i+k} - x_i} \end{aligned} \tag{19}$$

Evaluation via Horner's method requires n multiplications and n additions for a degree-n polynomial, which is optimal.

4.2.2 Application to LNS

For f^+ and f^-, sample points can be placed non-uniformly, with higher density near $x = 0$ where the functions curve sharply, and sparser sampling in the flat tail near $-\infty$. This is the key advantage over uniform splines: the same number of points can cover a much wider domain with controlled error.

For a piecewise approach, the domain is split into segments and a separate low-degree Newton polynomial is fitted to each. Segment boundaries are chosen adaptively based on the local curvature.

4.2.3 Memory and Evaluation Cost

Storing $n + 1$ points requires $2(n + 1)$ values: the x_i ones and the divided difference coefficients. For a degree-d polynomial on k segments, the total storage is $O(k \cdot d)$ values. Evaluation requires d multiply-adds per query.

4.2.4 Considerations

Newton's divided differences are numerically stable when using Horner's scheme, but the divided differences themselves can suffer from cancellation when computed in floating point for closely spaced points. For fixed-point arithmetic, careful scaling of the coefficients is needed to avoid overflow during evaluation.

4.3 Minimax Polynomial Approximation (Remez Algorithm)

4.3.1 Method

Rather than interpolating through sample points, minimax approximation finds the polynomial of degree n that minimises the maximum absolute error over the entire domain $[a, b]$:

$$p^*(x) = \arg\min_{p \in \mathcal{P}_n} \max_{x \in [a,b]} |f(x) - p(x)| \tag{20}$$

By the equioscillation theorem (Chebyshev), the optimal polynomial achieves its maximum error at exactly $n + 2$ alternating points. The Remez algorithm iteratively refines a candidate set of equioscillation points until convergence.

4.3.2 Application to LNS

For f^+, the domain is bounded $(-\infty, 0]$ and the function is smooth, so a single polynomial of moderate degree (typically 4–8) can achieve very small error over a truncated domain $\left[x_{\{m\in\}}, 0\right]$. The tail below $x_{\{m\in\}}$ is handled by clamping since $f^+(x) \approx 0$ for sufficiently negative x.

For f^-, the singularity at $x = 0$ prevents a single polynomial from covering the full domain. A piecewise approach is needed, or a change of variable such as $t = -x$ combined with approximating $f^-(-t)$ near $t = 0$ separately.

4.3.3 Memory and Evaluation Cost

A single degree-n minimax polynomial requires storing $n + 1$ coefficients. For lns16, a degree-6 polynomial fits in $7 \times 2 = 14$ bytes per function, compared to the 72–252 bytes required by spline tables. Evaluation via Horner's method costs n multiply-adds.

4.3.4 Considerations

Minimax approximation gives the theoretically optimal worst-case error for a given polynomial degree. The Remez algorithm is well established and its output is a set of fixed coefficients that can be computed offline and hardcoded. The main challenge for hardware is that the coefficients are real-valued and must be quantised to fixed point without degrading the approximation quality.

4.4 Piecewise Chebyshev Approximation

4.4.1 Method

Chebyshev approximation fits a truncated Chebyshev series to f on each subinterval $[a_k, b_k]$. The Chebyshev polynomials $T_{n(x)}$ form an orthogonal basis on $[-1, 1]$ and minimise the maximum norm among monic polynomials of degree n, making them near-optimal in the minimax sense.

For an interval $[a, b]$, the change of variable $t = \frac{2x-a-b}{b-a}$ maps to $[-1, 1]$, and the approximation is:

$$f(x) \approx \sum_{k=0}^{n} c_k T_{k(t)}, \quad c_k = \frac{2}{n} \sum_{j=1}^{n} f(x_j) T_{k(t_j)} \tag{21}$$

where x_j are the Chebyshev nodes. Truncating after $n + 1$ terms gives a near-minimax approximation.

4.4.2 Application to LNS

Piecewise Chebyshev is particularly suited to the subtraction function, where the singularity at $x = 0$ can be handled by assigning a dedicated segment $\left[x_{\{-1\}}, 0\right)$ with a higher-degree polynomial, while the smooth tail is covered by low-degree segments with wider spacing.

The coefficients of each segment can be quantised to fixed point and stored in a compact table. Unlike Newton's divided differences, Chebyshev coefficients have bounded growth and are less sensitive to quantisation.

4.4.3 Memory and Evaluation Cost

For k segments of degree d, storage is $k(d+2)$ values (coefficients plus boundary). Evaluation requires d multiply-adds per query using Clenshaw's recurrence, which is numerically stable and equivalent in cost to Horner's method.

4.5 Comparison of Methods

Method	Memory	Eval cost	Handles singularity	Notes
Linear spline (XF)	High	4 ops	Partial	Simple, synthesisable in 2 cycles
Linear spline (XMB)	High	1 MAC	Partial	Single cycle, used in hardware
Newton NDD	Low	d MACs	With piecewise	Non-uniform point placement
Minimax (Remez)	Very low	d MACs	With piecewise	Optimal worst-case error
Piecewise Chebyshev	Low	d MACs	Yes	Near-minimax, stable coefficients

Table 4: Qualitative comparison of approximation methods. d is polynomial degree per segment.

5 Extended Formats: lns32 and lns64

5.1 Motivation

For lns32 Q16.15, the argument to the correction functions $x = L_y - L_x$ ranges over 31 bits (excluding the sign). At the error level required for one-bit precision in Q16.15, a linear spline covering this domain would require tens of thousands of rows — far beyond any practical hardware budget.

For lns64 Q32.31, the situation is worse by a further factor of $2^{\{16\}}$.

The transition to compact polynomial approximations is therefore not optional for lns32 and lns64 — it is required.

5.2 Domain Analysis

For lns32, the full domain of f^+ is $\left(-2^{\{15\}}, 0\right]$ in fixed-point units (Q16.15). However, for $x < -52$ the value $2^x < 2^{\{-52\}}$, which is below the precision of 64-bit floating point. In practice, the correction function saturates to 0 for sufficiently negative x, and the effective domain requiring approximation is much narrower.

The effective domain can be defined as the range of x where $f^+(x) > 2^{\{-F\}}$, i.e., where the correction is larger than the LSB of the format. For lns32 Q16.15 with $F = 15$:

$$f^+(x) > 2^{\{-15\}} \Rightarrow \log_2(1 + 2^x) > 2^{\{-15\}} \Rightarrow 2^x > 2^{\{2^{\{-15\}}\}} - 1 \approx 2^{\{-15\}} \cdot \ln(2) \tag{22}$$

giving an effective lower bound around $x \approx -52$. Beyond this point the correction can be set to zero without loss of precision. The effective domain is therefore approximately $[-52, 0]$ regardless of the total exponent width, which makes compact polynomial approximation feasible even for lns32 and lns64.

The subtraction function f^- has a narrower effective domain: for $x < -52$, $f^-(x) \approx f^+(x)$ and both can be clamped. The critical region is near $x = 0$ where the singularity dominates.

5.3 Implications for Approximation

The observation that the effective domain is $O(52)$ in natural units (independent of format width) means that the same polynomial approximation can in principle be reused across lns8, lns16, lns32, and lns64 with only coefficient rescaling to account for the different fixed-point representations. This is a significant simplification and suggests a unified approximation strategy:

1. Compute the minimax or Chebyshev approximation for f^+ and f^- in real-valued arithmetic on $[-52, 0]$.
2. Quantise coefficients to the fixed-point precision of the target format.
3. Evaluate using Horner's method with integer arithmetic.

6 Evaluation Framework

6.1 Error Metrics

Approximation quality is measured by comparing the output of the approximation against the true value of $f^+(x)$ or $f^-(x)$ computed in 64-bit floating point. The following metrics are tracked:

- **Maximum absolute error (MAE)**: $\max|f(x) - \hat{f}(x)|$ in fixed-point units.
- **Maximum relative error (MRE)**: $\max|f(x) - \hat{f}(x)| f(x)|$.
- **Average absolute error**: $E\left[\!\left[f(x) - \hat{f}(x)\right]\!\right]$.
- **Error in bits**: $\log_2(\text{MAE}) - \log_2\left(2^{\{-F\}}\right) = \log_2(\text{MAE} \cdot 2^F)$, where F is the number of fractional bits.

An error of ≤ 1 bit means the approximation is off by at most one LSB of the format.

6.2 Test Input Distribution

Random inputs are sampled by drawing a uniformly random exponent from the format's representable range and converting to float via 2^e. To avoid overflow in addition tests, the exponent is capped:

Format	Exponent cap	Value range	Worst-case add
lns8 Q4.3	$[-3, +3]$	$[0.125, 8.0]$	$16.0 = 2^4$
lns16 Q8.7	$[-6, +6]$	$[0.015625, 64.0]$	$128.0 = 2^7$
lns32 Q16.15	$[-6, +6]$	$[0.015625, 64.0]$	$128.0 = 2^7$
lns64 Q32.31	$[-6, +6]$	$[0.015625, 64.0]$	$128.0 = 2^7$

Table 5: Test input ranges and exponent caps by format

The same cap is applied to bfloat16 in comparisons to ensure a fair benchmark. Without capping, both lns and floating-point formats exhibit large errors because their fractional bits are consumed by representing the integer part of large values, leaving no precision for sub-unit differences.

6.3 Benchmark Operations

For each format and approximation method, five operations are tested:

- **rt**: Round-trip conversion (float $\rightarrow$ LNS $\rightarrow$ float). Measures quantization error only.
- **mul**: Multiplication. Exact in LNS; tests exponent addition only.
- **div**: Division. Exact in LNS; tests exponent subtraction only.
- **add**: Addition. Exercises f^+ approximation.
- **sub**: Subtraction. Exercises f^- approximation, including near-cancellation cases.

Results are reported as min/avg/max absolute and relative error, and compared against bfloat16 as a 16-bit floating-point baseline for lns16.

7 Open Questions and Future Work

7.1 Approximation Method Selection

The primary open question is which approximation method provides the best trade-off between accuracy, memory, and hardware evaluation cost for each format. The following investigations are planned:

- Implement Newton's divided differences with adaptive point placement for lns8 and lns16 and compare against the existing XMB spline tables at the same memory budget.
- Apply the Remez algorithm to compute minimax polynomials of degree 4–8 for f^+ and f^- on the effective domain $[-52, 0]$, quantise to fixed point, and measure the resulting error.
- Implement piecewise Chebyshev approximation with special handling of the f^- singularity and evaluate on all four formats.

7.2 Fixed-Point Coefficient Quantisation

All polynomial methods require offline computation of coefficients in real arithmetic followed by quantisation to the target fixed-point format. The interaction between polynomial degree, coefficient wordlength, and approximation error under fixed-point evaluation is non-trivial and requires careful analysis, particularly for lns32 and lns64 where intermediate products in Horner evaluation may require extended-precision accumulators.

7.3 Hardware Implementation for lns32 and lns64

The LNSU design from previous work was optimised for lns16 with XMB splines and single-cycle latency. A polynomial evaluator for lns32 or lns64 will have a different area and latency profile. The degree of the polynomial and the wordlength of the coefficients will determine the number of DSP blocks required. An initial estimate suggests that a degree-6 evaluator in Q16.15 arithmetic would require 3–4 DSPs and would be pipelinable to maintain high throughput.

7.4 Conversion Operations

As noted in previous work, conversion between LNS and integer/float formats has not yet been implemented. For practical use of lns32 and lns64 in software simulation and hardware, these conversions are necessary. The conversion from float to LNS requires computing $\log_2(|x|)$ in fixed point, which can itself be approximated by a polynomial in the mantissa after extracting the IEEE-754 exponent.

7.5 Format Generalisation

The current library supports lns8, lns16 as templates parameterised by $l\angle N, I, Fr\angle$. Extending this to lns32 and lns64 requires ensuring that intermediate arithmetic (exponent differences, polynomial evaluation) does not overflow the available integer types. For lns64, the exponent field is 63 bits wide, requiring 128-bit intermediate arithmetic in software simulation.

Bibliography